# Reasoning about the ARM weakly consistent memory model

Nathan Chong
R&D

Samin Ishtiaq
CPU Design Verification

ARM Ltd
Cambridge, UK

## ABSTRACT

This paper describes a formalization of the ARM weakly consistent memory model: the architectural contract between parallel programs and shared memory multiprocessor implementations. We claim that a clean, unambiguous, and mechanically verifiable specification is a valuable resource for architects, micro-architects and programmers; it allows implementors to forge aggressive static (compiler) and dynamic (JIT, micro-architecture) machines to run code. We discuss the key construct of the ARM memory model, observability – the order in which memory accesses become visible to processors in a shared memory multiprocessor system – and examine its use in litmus tests.

## Categories and Subject Descriptors

B.3.2 [**Memory Structures**]: Design Styles—*Shared memory*; B.3.3 [**Memory Structures**]: Performance Analysis and Design Aids—*Formal models*; C.1.4 [**Processor Architectures**]: Parallel Architectures

## General Terms

Theory, Verification

## Keywords

ARM, Weakly consistent memory model

## 1. INTRODUCTION

The ARMv7 Architecture [1] extends the architecture to account for shared memory multiprocessor systems. Memory access order rules are defined to accommodate the increasing difficulty of ensuring linkage between the completion of memory accesses and the execution of instructions within a complex high-performance system, while also allowing simple systems and implementations to meet the criteria with predictable behaviour. The memory order model determines when side effects are guaranteed to be visible, and the requirements for memory consistency.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.
*MSPC'08* 2 March 2008, Seattle, WA, USA
Copyright 2008 ACM 978-1-60558-049-4 ...$5.00.

The purpose of this paper is to present a complete rational basis for the ARM memory model. Considering the need and utility of formalizing a memory model, our opinion is that there are two techniques that help in trying to make sense of a new topic: pictures and mathematics. Sometimes these combine (Turing machines, Feynman diagrams, category theory) to allow mathematicians to calculate with great efficacy and aesthetics. Pictures and mathematics are clear, unambiguous, lucid and reliable; they are evocative models that allow the user to be absolute about the correctness of their calculations. A presentation in this style is a valuable adjunct to the humans-and-words mainstream architectural repository that we currently have. It helps in communicating the design amongst those interested. It also means that reasoning about memory access order need not turn into an exercise in exegesis.

The formalization of the ARM memory model that we illustrate here is driven by practical needs. It is important to a large number of users (those involved in code translation, thread libraries, OS kernels, CPU design and verification) that the memory model is explicit, and its consequences well understood. The formalization is also driven by the ideal that a well-specified hardware forms the base of a verified stack [14] with operating systems[6], virtual machines[10] and compilers[12] built over it. A clear specification means that we can conceive of verifying when an implementation complies with the architecture.

Formalization of memory models has a history. Collier at IBM [3] established the tradition of reasoning from litmus tests, and there have been formalizations for Intel IA-64[9] and Itanium[16], Sparc[8], and Alpha[4, 7] memory models. Even in the software world, where the economics of sending out patches is more favourable, the Java Memory Model (JMM) originally had fundamental bugs in it that a more rigourous presentation of JSR133 put to rest [13]. Recently, there has been interest in memory models for C++ and Microsoft native code platforms [2, 15].

The model we illustrate here is in axiomatic style. There are no events such as clocks (certainly there are no global clocks). We model time abstractly via a per-processor partial ordering of accesses (Section 2, Observability). We do not have space to present the full model, but just show the complexity of the observability and barrier concepts.

The entire development reported here has been modelled in the Calculus of Inductive Constructions, the type theory of the theorem prover Coq[5]. The basic data structure we work with are graphs whose nodes are memory accesses. Our working style is to define functions to compute results and then prove that these results have desired properties, rather

than working exclusively with predicates.

## 2. OBSERVABILITY

Under a weakly consistent memory model, the meaning of the program depends upon the order in which accesses appear to be executed by an observer (such as a processor). More, the apparent order of accesses being executed may be different for each observer. Observability is defined in the ARM architecture in terms of local time:

- A write to a location in memory is said to be observed by an observer $Pn$ when a subsequent read of the location by $Pn$ would return the value written by the write.
- A read of a location in memory is said to be observed by an observer $Pm$ when a subsequent write to the location by $Pm$ would have no effect on the value returned by the read.

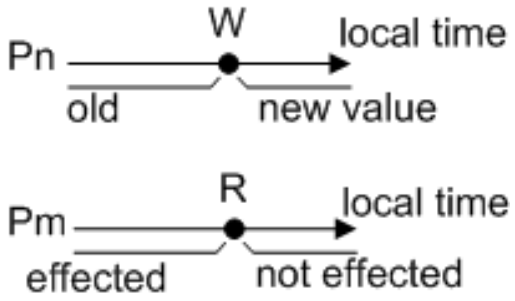



**Figure 1: Pn (Pm) observes a write (read)**

Read observed can be seen to be the dual of write observed. Consider a simple example, where P1 (P2) writes (reads) a shared variable. If P2 reads the old (new) value, then P1 (P2) is said to observe the read (write). This duality is used in the inductive cases in the definition of barriers below.

Observation of an access by an observer can be thought of as a local timestamp indicating the point at which the access takes effect. Observers are not required to agree on the ordering of accesses. It is incorrect to reason about accesses that lie on different observers timelines; for instance, different processors' observations cannot be composed.

We do not model observability explicitly. Instead we abstract the ordering by considering a partial order over accesses for each observer. We write $a <_{ob(Pn)} b$ to denote that processor $Pn$ observed access $a$ before access $b$.

## 3. BARRIER RULES

The ARM DMB instruction is a data memory barrier; the software programmer can use DMBs to force the observability order of accesses.

When a processor $Pe$ executes a DMB we require that all processors observe all Group A accesses before any Group B accesses:

*Group A* contains:

**base:** all memory accesses that are observed by $Pe$ before the DMB instruction. This includes accesses performed by $Pe$.

**step:** all loads that have been observed by any given observer $Py$ before $Py$ has performed a memory access that is a member of Group A.

*Group B* contains:

**base:** all memory accesses by $Pe$ that occur in program order after the DMB instruction.

**step:** all memory accesses by any given observer $Px$ that can only occur after $Px$ has observed a store that is a member of Group B.

We write $a <_{po} b$ to denote that access $a$ is before access $b$ in program order, and $a <_{all} b$ to denote that all processors observe access $a$ before access $b$.

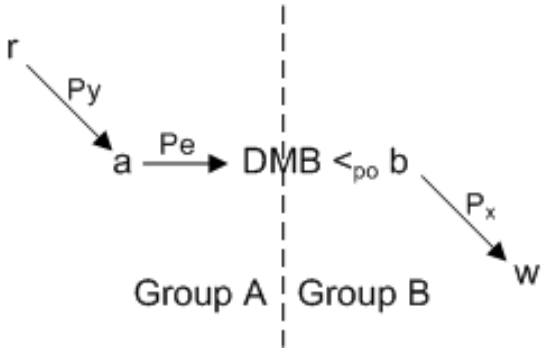



**Figure 2: Group A and Group B for a DMB are defined using program order and observability**

DMB does not require global synchronization amongst processors. The ARM DSB (Data Synchronization Barrier) instruction has this property additionally: that all Group A accesses are completed (observed by all observers) before any Group B access completes.

## 4. LITMUS TESTS

Litmus tests are small multiprocessor programs used to demonstrate particular features of a memory consistency model. It is often the case that a litmus test will have a counter-intuitive outcome on a weakly consistent memory model. For example, the tests that we consider next are not permissible under sequential consistency [11]. A litmus test is permissible if there exists an observability order (there could be several) which has the following properties:

*DAG* The order is a strict partial order on the litmus test. No processor must have a cycle in its observability graph.

*Uniprocessor* Uniprocessor order rules are given by data hazards, address dependence (when registers are shared), and control dependence. If access $a$ is ordered before access $b$ (both executed on Processor $Pn$) by a uniprocessor rule, then $a <_{obs(Pn)} b$.

*Write Serialization* All writes to the same location have a total order which is respected by all processors. That is, for any two writes, $w$ and $w'$, to the same location then either $w <_{all} w'$ or $w' <_{all} w$.

*Barrier* For every barrier $dmb$, if $a \in GroupA(dmb)$ and $b \in GroupB(dmb)$, then $a <_{all} b$.

*Value Requirement* Every read in a litmus test is bound to either the initial value of the location, or to the value of the last write observed to the same location.

*Final State* A symbolic simulation of the litmus test should leave memory in the same state as given by the final reads.

Our Coq-based litmus test flow is currently entirely manual: to show permissibility of a litmus test, we guess a candidate order and mechanically prove that this order is permissible. To prove that a litmus test is not permissible, we need to show that, for all candidate partial orders generated by the litmus test, none is permissible. This is practical only for small sized litmus tests, and we further plan to automate this proof search.

We use the following notation for accesses: (`W x v`) is a write access to location x with value v; (`R x v`) is a read access from location x which returns the value v; and `B` is a barrier access generated from a DMB instruction. In all of the following tests the initial value of all memory locations is 0. In the graphs, we use a labelled edge $a \xrightarrow{Pn} b$ to denote $a <_{ob(Pn)} b$.

## 4.1 Causal consistency issues with multiple observers

Consider the following test, which is similar to Intel's 'Stores are transitively visible' example[9, Test2.5]. We assume a control dependency between b1 and b2; and an address dependency between c1 and c2.

| p1 | | p2 | | p3 | |
|---|---|---|---|---|---|
| a1: | W x 1 | b1: | R x 1 | c1: | R y 1 |
| | | b2: | W y 1 | c2: | R x 0 |

In the absence of barriers, P3:x=0 is permissible. That is, P3 is not guaranteed to see the write from P1 and the write from P2 in any particular order (despite the fact that the store from P2 can only happen after P2 has observed the store from P1).

The candidate ordering given in Figure 3 shows this litmus test permissible. Write observed gives us the edges $\{a1 <_{p2} b1, b2 <_{p3} c1\}$; Uniprocessor gives us the edges $\{b1 <_{p2} b2, c1 <_{p3} c2\}$; Read observed gives us the edge $c2 <_{p3} a1$. The graph has no cycle which traverses the same processor's observability order.

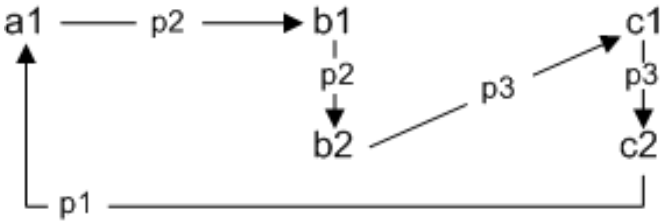



**Figure 3: Stores are not transitively ordered**

## 4.2 Multiple observers of writes to multiple locations

Consider the following test, which shows that different observers can observe writes to different locations in different orders.

| p1 | | p2 | | p3 | |
|---|---|---|---|---|---|
| a1: | W x 1 | b1: | R y 1 | c1: | R x 1 |
| a2: | W y 1 | b3: | R x 0 | c3: | R y 0 |

In the absence of barriers this test is permissible, which means that P2 and P3 have observed the stores from P1 in different orders. Figure 4 gives a candidate ordering that shows this result.

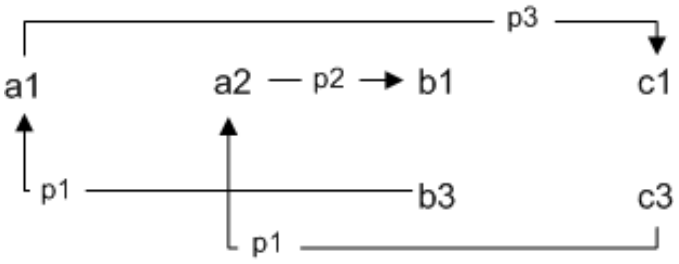



**Figure 4: Writes to different locations can be observed in different orders**

## 4.3 Resolving with barriers

We can force a final P3:y=1 by inserting barriers between b1 and b3 and c1 and c3. This guarantees that P2 and P3 observe P1's write to y (a2) before their respective reads. That is, the test as given above is not permissible.

We start with the same candidate ordering as the previous test. The barriers add the following edges to the ordering:

| Barrier | Group A | Group B |
|---|---|---|
| b2 | b1, a2, c3 | b3 |
| c2 | c1, a1, b3 | c3 |

The calculation for $GroupA(b2)$ is:

- $b1 \in GroupA(b2)$ by GroupA's base rule, as $b1 <_{p2} b2$;
- $a2 \in GroupA(b2)$ also by GroupA's base rule, as $a2 <_{p2} b2$;
- $c3 \in GroupA(b2)$ by GroupA's step rule, as $c3 <_{p1} a2$. Recall that the rule is: 'all loads (c3) that have been observed by any given observer $Py$ (p1) before $Py$ (p1) has performed a memory access that is a member of Group A (a2)'.

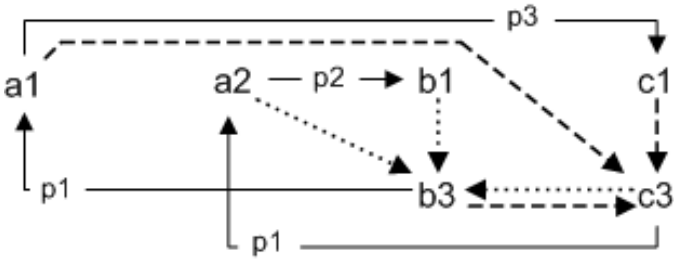



**Figure 5: Barrier edges**

Figure 5 illustrates the extra edges added by the barriers b2 (dotted arrows) and c2 (dashed arrows). These two barriers create a cycle $c3 <_{all} b3$ and $b3 <_{all} c3$. This graph then fails the DAG condition of permissibility.

## 5. FURTHER WORK

We are currently working on two extensions to the work described in this paper.

First, in this paper, DMB instructions were sufficient to resolve the issues in our litmus tests because we have not considered out-of-memory communication such as interrupts. In some message passing systems, it is common for one observer to update memory and then send an interrupt to a second observer to signal that the data is ready. In such cases, a DSB is required to ensure that the update to memory completes before the interrupt is sent.

The difference between these two barrier instructions can be understood operationally as: a DMB is a fire and forget operation; a DSB requires acknowledgement, the synchronization, that all Group A accesses have completed.

Second, there are several completeness questions we need to answer. How does the axiomatic model fit in with the operational one? Can we prove that appending DMBs to each instruction results in a sequentially consistent execution? And what is the precise embedding of the ARM model into Alpha, Intel, JMM? Part of the problem here is that precise, comparable descriptions of these models are not readily available. Some comparisons would mainly be of academic interest (we don't plan to build a multi-core where ARM and Alpha need to share the memory system), but others are more relevant: the Java mapping would tell us what architectural support a programming language requires, and so would be of possible use to JVM porters.

## 6. ACKNOWLEDGEMENTS

We are grateful to Richard Grisenthwaite for feedback and corrections on this work.

## 7. REFERENCES

[1] ARM. ARM v7A+R Architectural Reference Manual. Available from ARM Ltd.

[2] Hans-J Boehm. Threads cannot be implemented as a library. In *PLDI '05*, 2005.

[3] William W Collier. *Reasoning about parallel architectures*. Prentice-Hall Inc., 1992.

[4] Compaq. *Alpha Architecture Handbook*. Compaq, 1998.

[5] The Coq Development Team. *The Coq proof assistant reference manual*. LogiCal Project, 2006. Version 8.1.

[6] K Elphinstone, G Klein, P Derrin, T Roscoe, and G Heiser. Towards a practical, verified kernel. In *Proc. 11th Workshop on Hot Topics in Operating Systems*, 2007.

[7] Mike Gordon. A formalization of a simplified subset of the Alpha shared memory model, 1994. Manuscript.

[8] S Hangal, D Vahia, C Manovit, J-Y J Lu, and S Narayanan. TSOtool: A program for verifying memory systems using the memory consistency model. In *ISCA*, 2004.

[9] Intel. Intel 64 Architecture Memory Ordering White Paper. SKU 318147-001, August 2007.

[10] G Klein and T Nipkow. A machine-checked model for a Java-like language, virtual machine and compiler. *TOPLAS*, 28(4), 2006.

[11] Leslie Lamport. How to make a correct multiprocess program execute correctly on a multiprocessor. *IEEE Trans. Comput.*, 46(7):779–782, 1997.

[12] Xavier Leroy. Formal certification of a compiler back-end. In *Proc of POPL*, 2006.

[13] Jeremy Manson, William Pugh, and Sarita V Adve. The Java Memory Model. In *Proc of POPL*, 2006.

[14] JS Moore. A Grand Challenge Proposal for Formal Methods: A Verified Stack. In *Formal Methods at the Crossroads*, 2003.

[15] H Sutter. Prism: A Principle-Based Sequential Memory Model for Microsoft Native Code Platforms. March 2007. Working draft v0.9.4.

[16] Y Yang, G Gopalakrishnan, G Lindstrom, and K Slind. Analyzing the Intel Itanium Memory Ordering Rules Using Logic Programming and SAT. In *Proc of CHARME*, 2003.